
Madár Patrik

Kalman szűrők

Budapest, 2026

Contents

1	Kalman szűrő	3
1.1	Levezetés	3
1.2	Implementáció	5
2	Kiterjesztett Kalman szűrő	6
2.1	Levezetés	6
2.2	Implementáció	7
2.3	Példa	8

Chapter 1

Kalman szűrő

1.1 Levezetés

A Kalman szűrő a Bayes szűrő egy speciális esete lineáris Gauss-rendszerekre. Fő erénye a Bayes szűrővel szemben hogy az ott predikciós lépésben lévő integrált képes zárt alakban kiszámolni, evvel csökkentve a számításigényt. A Bayes szűrő mintájára a rejtett állapotokat x_t -vel, a megfigyelteket y_t -vel jelölve, a modell alakja

$$x_t = A_t x_{t-1} + B_t u_t + w_t$$

$$y_t = C_t x_t + n_t,$$

ahol A_t, B_t, C_t konstant mátrixok minden t -re, $x_0 \sim N(\mu_0, \Sigma_0)$ a kezdőállapot (azaz $bel(x_0) = N(x_0; \mu_0, \Sigma_0)$), u_t control input, $w_t \sim N(0, R_t)$ folyamat zaj, $n_t \sim N(0, Q_t)$ megfigyelési zaj.

A predikciós lépés

$$\overline{bel}(x_t) = \int p(x_t | x_{t-1}, u_t) \cdot bel(x_{t-1}) dx_{t-1}$$

integráljában lévő feltételes valószínűség x_{t-1}, u_t feltételeit rögzítve a $w_t \sim N(0, R_t)$ normális eloszlás $A_t x_{t-1} + B_t u_t$ -vel vett eltoltját kapjuk, így ezen feltételezéssel

$$p(x_t | x_{t-1}, u_t) \sim N(A_t x_{t-1} + B_t u_t, R_t),$$

melyből

$$\overline{bel}(x_t) = \int N(x_t, A_t x_{t-1} + B_t u_t, R_t) \cdot N(x_{t-1}, \mu_{t-1}, \sigma_{t-1}) dx_{t-1}$$

Ebből

$$\overline{bel}(x_t) = \eta \cdot \int e^{-L_t} dx_{t-1},$$

ahol

$$L_t = \frac{1}{2}(x_{t-1} - \mu_{t-1})\Sigma_{t-1}^{-1}(x_{t-1} - \mu_{t-1}) + \frac{1}{2}(x_t - A_t x_{t-1} - B_t u_t)R_t^{-1}(x_t - A_t x_{t-1} - B_t u_t)$$

Célunk hogy adjunk $L_t = L_t^1(x_t) + L_t^2(x_{t-1}, x_t)$ kvadratikus alakok dekompozícióját, ekkor ugyanis

$$\overline{bel}(x_t) = \eta_1 e^{-L_t^1(x_t)} \int e^{-L_t^2(x_{t-1}, x_t)} dx_{t-1} = \eta_2 e^{-L_t^3(x_t)}$$

L_t -ben az x_t lerögzítésével kapott kvadratikus alaknak x_{t-1} szerinti első és második deriváltja megadja az általa meghatározott normális eloszlás várható értékét és görbületét

($L_t^1(x_t)$ ekkor konstans, annak deriváltja 0): a várható érték az első derivált minimumhelye, a görbület inverze éppen a kovariancia:

$$\begin{aligned} 0 &= \frac{\delta L_t}{\delta x_{t-1}} = \Sigma_{t-1}^{-1}(x_{t-1} - \mu_{t-1}) - A_t^T R_t^{-1}(x_t - A_t x_{t-1} - B_t u_t) \\ &\Rightarrow (\Sigma_{t-1}^{-1} + A_t^T R_t^{-1} A_t) x_{t-1} = \Sigma_{t-1}^{-1} \mu_{t-1} + A_t^T R_t^{-1}(x_t - B_t u_t) \\ &\Rightarrow \mu_t = \Psi_t^{-1}(\Sigma_{t-1}^{-1} \mu_{t-1} + A_t^T R_t^{-1}(x_t - B_t u_t)) \end{aligned}$$

Mivel az első derivált lineáris x_{t-1} -ben, így a második derivált éppen x_{t-1} együtthatója, azaz

$$\Sigma_t^{-1} = \Psi_t = \frac{\delta^2 L_t}{\delta^2 x_{t-1}} = \Sigma_{t-1}^{-1} + A_t^T R_t^{-1} A_t$$

Ebből

$$\Sigma_t = (\Sigma_{t-1}^{-1} + A_t^T R_t^{-1} A_t)^{-1}$$

Ekkor a felbontás

$$\begin{aligned} L_t^2(x_{t-1}, x_t) &= \frac{1}{2}(x_{t-1} - \mu_t)^T \Psi_t (x_{t-1} - \mu_t) \\ \Rightarrow L_t^1(x_t) &= L_t - \frac{1}{2}(x_{t-1} - \mu_t)^T \Psi_t (x_{t-1} - \mu_t) \end{aligned}$$

Az egyszerű alaknál ekkor az együttható szorozódik $\int e^{L_t^2(x_{t-1}, x_t)} = \sqrt{2\pi \Psi_t}$ -vel. Hasonlóan $L_t^1(x_t)$ kvadratikus alak, erre a deriválásokat elvégezve

$$\overline{\mu}_t = A_t \mu_{t-1} + B_t u_t$$

$$\overline{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t,$$

melyből

$$\overline{bel}(x_t) = N(x_t; \overline{\mu}_t, \overline{\Sigma}_t)$$

Erre a Bayes szűrő korrekciós lépése:

$$bel(x_t) = \eta p(y_t | x_t) \overline{bel}(x_t) = \eta N(y_t; C_t x_t, Q_t) \cdot N(x_t, \overline{\mu}_t, \overline{\Sigma}_t) = \eta e^{-J_t}$$

valamilyen J -re. Ebből előbbi módon deriválással

$$\Sigma_t = (C_t^T Q_t^{-1} C_t + \overline{\Sigma}_t^{-1})^{-1}$$

$$\mu_t = \overline{\mu}_t + \Sigma_t C_t^T Q_t^{-1} (y - C_t \overline{\mu}_t)$$

Itt a

$$K_t = \Sigma_t C_t^T Q_t^{-1} = \overline{\Sigma}_t C_t^T (C_t \overline{\Sigma}_t C_t^T + Q_t)^{-1}$$

értéket Kalman erősítésnek hívjuk. Ezzel

$$\Sigma_t = (I - K_t C_t) \overline{\Sigma}_t$$

$$\mu_t = \overline{\mu}_t + K_t (y_t - C_t \overline{\mu}_t)$$

1.2 Implementáció

Előbbiekben levezetett formulákkal megadható a Kalman szűrő vázlatos implementációja:

```
1 import numpy as np
2
3 class KalmanFilter():
4     def __init__(self, F, B, G, Q, R, mu, Sigma):
5         self.F = F #State transition
6         self.B = B #Control
7         self.G = G #Measurement
8         self.Sigma = Sigma #Estimate error covariance
9         self.R = R #Process noise covariance
10        self.Q = Q #Observation noise covariance
11        self.mu = mu
12
13    def predict(self, u=0):
14        self.mu = self.F @ self.mu + self.B @ u if isinstance(u, np.
15        ndarray) else self.F @ self.mu + self.B * u
16        self.Sigma = self.F @ self.Sigma @ self.F.T + self.R
17
18    def correction(self, y):
19        z = y - self.G @ self.mu #Innovation
20        S = self.G @ self.Sigma @ self.G.T + self.Q #Innovation
21        covariance
22        K = self.Sigma @ self.G.T @ np.linalg.inv(S)
23
24        self.mu = self.mu + K @ z
25        self.Sigma = (np.eye(self.Sigma.shape[0]) - K @ self.G) @
26        self.Sigma
27
28    def return self.mu
```

Chapter 2

Kiterjesztett Kalman szűrő

2.1 Levezetés

A Kalman szűrő kifejezetten lineáris Gauss rendszerek megoldásait szolgáltatja, így gyakran nem alkalmazható. A Kalman szűrő Gauss tulajdonságával és megfelelő valószínűségi változóival ha a modell

$$\begin{aligned}x_t &= f(x_{t-1}, u_t) + w_t \\ y_t &= g(x_t) + n_t\end{aligned}$$

alakú, ahol f, g nem lineáris függvények, akkor az egyszerű esetben levezetett L_t nem lesz kvadratikus x_{t-1} -ben, így nem lesz megadható az ott szereplő $L_t = L_t^1(x_t) + L_t^2(x_{t-1}, x_t)$ felbontás, amiből $\overline{bel}(x_k)$ normális eloszlása sem következik.

Adódik az ötlet, hogy vegyük a függvények Taylor-sorainak elsőrendű közelítéseit, majd arra alkalmazzuk a lineáris esetre megoldást adó Kalman szűrő algoritmusát.

A Bayes szűrő predikciós lépése ekkor

$$\overline{bel}(x_t) = \int N(x_t; f(x_{t-1}, u_t), R_t) \cdot N(x_{t-1}; \mu_{t-1}, \Sigma_{t-1}) dx_{t-1}$$

A Taylor sor

$$f\left(\begin{smallmatrix} x_{t-1} \\ u_t \end{smallmatrix}\right) \approx f(a) + J_f(a) \cdot \left(\begin{smallmatrix} x_{t-1} \\ u_t \end{smallmatrix} - a\right),$$

ahol $a = \left(\begin{smallmatrix} \mu_{t-1} \\ u_t \end{smallmatrix}\right)$ módon megadott elsőrendű közelítését véve

$$\begin{aligned}f\left(\begin{smallmatrix} x_{t-1} \\ u_t \end{smallmatrix}\right) &= f\left(\begin{smallmatrix} \mu_{t-1} \\ u_t \end{smallmatrix}\right) + J_f\left(\begin{smallmatrix} x_{t-1} - \mu_{t-1} \\ u_t - u_t \end{smallmatrix}\right) \\ \Rightarrow f\left(\begin{smallmatrix} x_{t-1} \\ u_t \end{smallmatrix}\right) &= f\left(\begin{smallmatrix} \mu_{t-1} \\ u_t \end{smallmatrix}\right) + F_{t-1}(x_{t-1} - \mu_{t-1}),\end{aligned}$$

ahol $F_{t-1} = \left.\frac{\delta f}{\delta x_{t-1}}\right|_{\left(\begin{smallmatrix} \mu_{t-1} \\ u_t \end{smallmatrix}\right)}$

Tehát

$$\overline{bel}(x_t) \approx \int N(x_t; f(x_{t-1}, u_t) + F_{t-1}(x_{t-1} - \mu_{t-1}), R_t) \cdot N(x_{t-1}; \mu_{t-1}, \Sigma_{t-1}) dx_{t-1}$$

Az ebből kapott L_t kvadratikus x_{t-1} -ben, erre alkalmazható a lineáris esetre vonatkozó Kalman szűrő, melyből

$$\begin{aligned}\overline{\mu}_t &= f\left(\begin{smallmatrix} \mu_{t-1} \\ u_t \end{smallmatrix}\right) \\ \overline{\Sigma}_t &= F_{t-1}\Sigma_{t-1}F_{t-1}^T + R_t \\ \Rightarrow \overline{bel}(x_t) &\approx N(x_t; f\left(\begin{smallmatrix} \mu_{t-1} \\ u_t \end{smallmatrix}\right), F_{t-1}\Sigma_{t-1}F_{t-1}^T + R_t)\end{aligned}$$

A Bayes szűrő korrekciós lépése itt

$$\begin{aligned} bel(x_t) &= \eta_1 p(y_t | x_t) \overline{bel}(x_t) \\ \Rightarrow bel(x_t) &= \eta_1 N(y_t; g(x_t), Q_t) \cdot N(x_t; \overline{\mu}_t, \overline{\Sigma}_t) \end{aligned}$$

Előbbiekhez hasonlóan g -nek is véve az elsőrendű Taylor-közelítését

$$g(x_t) = g(\overline{\mu}_t) + G_t(x_t - \mu_t),$$

ahol $G_t = \frac{\delta g}{\delta x} |_{x=\overline{\mu}_t}$

A Kalman szűrő számolásaival

$$\begin{aligned} K_t &= \overline{\Sigma}_t G_t^T (G_t \overline{\Sigma}_t G_t^T + Q_t)^{-1} \\ \mu_t &= \overline{\mu}_t + K_t(y_t - g(\overline{\mu}_t)) \\ \Sigma_t &= (I - K_t G_t) \overline{\Sigma}_t \end{aligned}$$

Figyeljük meg hogy a korrekciós lépésnél ugyanazt az alakot kaptuk mint a lineáris rendszerekre vonatkozó Kalman szűrő korrekciós lépésénél, avval a különbséggel hogy a konstans C_t helyett a g függvényt alkalmazzuk $\overline{\mu}_t$ -ra.

2.2 Implementáció

```
1 import numpy as np
2
3 class ExtendedKalmanFilter():
4
5     def __init__(self, f, F_jacobian, g, G_jacobian, Q, R, mu, Sigma):
6         self.f = f
7         self.F_jacobian = F_jacobian
8         self.g = g
9         self.G_jacobian = G_jacobian
10        self.Q = Q
11        self.R = R
12        self.mu = mu
13        self.Sigma = Sigma
14
15    def predict(self, u=0):
16
17        F = self.F_jacobian(self.mu, u)
18
19        self.mu = self.f(self.mu, u)
20
21        self.Sigma = F @ self.Sigma @ F.T + self.R
22
23        return self.mu
24
25    def correction(self, y):
26
27        G = self.G_jacobian(self.mu)
28
29        z = y - self.g(self.mu) # Innovation
30        S = G @ self.Sigma @ G.T + self.Q #Innovation covariance
31
32        K = self.Sigma @ G.T @ np.linalg.inv(S)
33
```

```

34         self.mu = self.mu + K @ z
35         self.Sigma = (np.eye(self.Sigma.shape[0]) - K @ G) @ self.
Sigma
36
37         return self.mu, self.Sigma

```

2.3 Példa

A kiterjesztett Kalman szűrő alkalmazható reakciókinetikai paraméterbecsléshez is. Tekintsünk egy k sebességállandós elsőrendű (n -edrendű is lehet, abban az esetben a koncentrációt n -edik kitevővel vesszük)



reakciót, ahol az $[A]$ koncentráció változásának differenciálegyenlete

$$\frac{d[A]}{dt} = -k[A]$$

Zajos mérésekből szeretnénk becslést adni k -ra. Feltesszük hogy k időben állandó. Ekkor

$$x = \begin{pmatrix} [A] \\ k \end{pmatrix},$$

továbbá

$$f(x) = \begin{pmatrix} [A] - \Delta t \cdot k \cdot [A] \\ k \end{pmatrix},$$

aminek Jacobi mátrixa

$$F_J = \begin{pmatrix} 1 - \Delta t \cdot k & -\Delta t \cdot [A] \\ 0 & 1 \end{pmatrix}$$

Ezek Python kódja:

```

1 dt = 0.1
2
3 def f(x, u=0):
4     A, k = x[0], x[1]
5     A_next = A - dt * k * A
6     k_next = k
7     return np.array([A_next, k_next])
8
9 def F_jacobian(x, u=0):
10    A, k = x[0], x[1]
11
12    return np.array([
13        [1.0 - dt * k, - dt * A],
14        [0.0, 1.0]
15    ])
16
17 def g(x):
18     return np.array([x[0]])
19
20 def G_jacobian(x):
21     return np.array([[1.0, 0.0]])

```

Most inicializáljuk a feladatot. Ehhez megadunk a rendszer szerint megfelelő valós és megfigyelt értékeket, a becslést ezek alapján végezzük:

```

1 np.random.seed(42)
2 N_steps = 100
3
4 x_real = np.array([10.00, 0.3]) #mol/dm^3, 1/s
5
6 real_states = []
7 measurements = []
8
9 process_noise_std = 0.01
10 measurement_noise_std = 0.2
11
12 for _ in range(N_steps):
13
14     x_real = f(x_real)
15     x_real[0] += np.random.normal(0, process_noise_std)
16
17     z = g(x_real) + np.random.normal(0, measurement_noise_std)
18
19     real_states.append(x_real.copy())
20     measurements.append(z)
21
22 real_states = np.array(real_states)
23 measurements = np.array(measurements)
24
25 #Initialization
26
27 mu_0 = np.array([8.0, 0.05])
28 Sigma_0 = np.diag([2.0, 0.5])
29
30 R = np.diag([1e-4, 1e-5]) #Process noise
31 Q = np.diag([measurement_noise_std**2]) #Measumenent noise
32
33 ekf_of_reaction = ExtendedKalmanFilter(
34     f=f,
35     F_jacobian = F_jacobian,
36     g=g,
37     G_jacobian=G_jacobian,
38     Q=Q,
39     R=R,
40     mu=mu_0,
41     Sigma=Sigma_0
42 )

```

Bár a folyamatban k konstans, minimális kovariancia ($R_{22} = 10^{-5}$) hozzáadásával ügyeltünk arra hogy az algoritmus ne ragadjon be egy hibás kezdeti becslésnél, az $[A]$ -hoz tartozó $R_{11} = 10^{-4}$ folyamat zaj a szimulációban alkalmazott 0.01-es zajszintet tükrözi. Erre alkalmazva a kiterjesztett Kalman szűrő algoritmusát, a kezdőértékből indulva, minden lépésben végrehajtjuk a Bayes szűrő predikciós majd a megfigyelésekből kapott korrekciós lépését:

```

1 estimated_states = []
2 estimated_covariances = []
3
4 for y in measurements:
5
6     ekf_of_reaction.predict()
7
8     mu, Sigma = ekf_of_reaction.correction(y)
9
10    estimated_states.append(mu.copy())

```

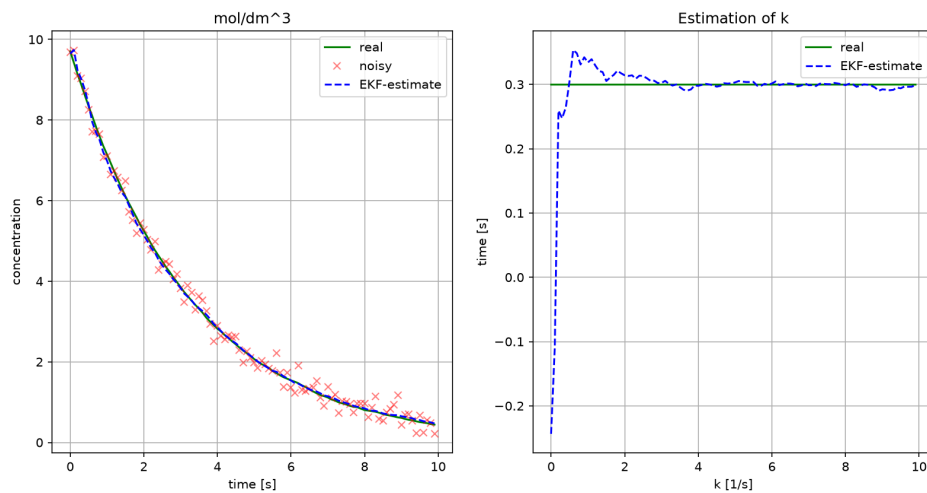


Figure 2.1: Bal oldalt koncentráció, jobb oldalt k becslése EKF-fel, $dt = 0.1$

```

11     estimated_covariances.append(Sigma.copy())
12
13     estimated_states = np.array(estimated_states)

```